Central de Chamados de Suporte

Instruções da atividade prática em Java

Objetivo

Construir, em Java, o núcleo de uma central de chamados de suporte: uma fila de prioridade com heap e duas ordenações simples para comparar estabilidade.

Cada chamado tem um id (ordem de chegada: o 1 chegou primeiro) e uma prioridade de 1 (baixa) a 5 (urgente). Atende-se primeiro a maior prioridade; em caso de empate, atende-se quem chegou antes.

Instruções gerais

• Atividade individual, realizada no computador, em Java.

• É proibido usar Arrays.sort, Collections.sort e PriorityQueue: os algoritmos devem ser escritos por você.

• A consulta é permitida apenas aos slides das aulas 06, 07 e 08.

• Entrega: Chamado.java, FilaPrioridade.java e Main.java.

• Vale 1 ponto extra na prova.

Código inicial

Arquivo Chamado.java

```
public class Chamado {
    int id;           // ordem de chegada
    int prioridade;   // 1 (baixa) a 5 (urgente)

    public Chamado(int id, int prioridade) {
        this.id = id;
        this.prioridade = prioridade;
    }

    @Override
    public String toString() {
        return "#" + id + "(P" + prioridade + ")";
    }
}
```

Arquivo Main.java

```
public class Main {
    static Chamado[] criarChamados() {
        int[] prioridades = {3, 3, 5, 4, 5, 1, 4, 2};
        Chamado[] chamados = new Chamado[prioridades.length];

        for (int i = 0; i < prioridades.length; i++) {
            chamados[i] = new Chamado(i + 1, prioridades[i]);
        }
        return chamados;
    }

    public static void main(String[] args) {
        Chamado[] chamados = criarChamados();
        // As Partes 1, 2 e 3 entram aqui
    }
}
```

Parte 1 – Fila de prioridade com heap máximo

Implemente FilaPrioridade com um heap máximo guardado em um vetor de Chamado, sem ponteiros e sem estruturas prontas. Para o nó no índice i: filho esquerdo em 2i+1, filho direito em 2i+2 e pai em ■(i–1)/2■.

Estrutura inicial de FilaPrioridade.java

```
public class FilaPrioridade {
    private Chamado[] heap;
    private int tamanho;

    public FilaPrioridade(int capacidade) {
        heap = new Chamado[capacidade];
        tamanho = 0;
    }

    // Item 1.1
    private boolean temPrioridadeSobre(Chamado a, Chamado b) {
        // TODO
    }

    // Item 1.2
    public void inserir(Chamado c) {
        // TODO: coloque no próximo espaço livre e faça o chamado subir
    }

    // Item 1.3
    public Chamado removerMaior() {
        // TODO: retire a raiz, leve o último para a raiz e faça-o descer
    }

    public void imprimirVetor() {
        for (int i = 0; i < tamanho; i++) {
            System.out.print(heap[i] + " ");
        }
        System.out.println();
    }
}
```

Item	O que fazer
1.1	temPrioridadeSobre(a, b): devolve true se a tem prioridade maior que b. Em empate, devolve true se a chegou antes (menor id).
1.2	inserir (com um método auxiliar subir): coloca o chamado na próxima posição livre do vetor e o troca com o pai enquanto ele tiver prioridade sobre o pai. Avise se a fila estiver cheia.
1.3	removerMaior (com um método auxiliar descer): devolve o chamado mais urgente, move o último do vetor para a raiz e o troca sempre com o filho de maior prioridade, considerando só filhos que estão dentro do heap. Fila vazia devolve null.
1.4	No main: insira os 8 chamados na ordem de chegada, chame imprimirVetor() e depois remova todos, imprimindo a ordem de atendimento.

Parte 2 – Estabilidade: Insertion Sort x Selection Sort

Ordene o vetor chamados por prioridade, da maior para a menor, comparando somente a prioridade (o id não entra na comparação). Uma ordenação estável mantém a ordem de chegada entre chamados de mesma prioridade; uma instável pode desrespeitá-la. Ordene sempre uma cópia: `Chamado[] copia = chamados.clone();`

Item	O que fazer
2.1	insertionSort(Chamado[] v), decrescente por prioridade. Conte os deslocamentos de elementos e imprima o total.
2.2	selectionSort(Chamado[] v), decrescente por prioridade. Para escolher o maior, use comparação estrita (>); troque só quando for necessário e conte as trocas.
2.3	Imprima os id na ordem final da fila da Parte 1, do Insertion Sort e do Selection Sort. Em comentário, aponte quais chamados de mesma prioridade ficaram fora da ordem de chegada.

Observação: o material acima organiza as instruções fornecidas e preserva os trechos de código inicial; os métodos marcados como TODO devem ser implementados pelo aluno.